

CLAUDE.md Best Practices

Tổng hợp từ 15+ nguồn uy tín — official docs, community consensus, và evidence-based research trên 1,650+ sessions.

Synthesized Oct 2026 · Claude Code v1.x+ · 15+ sources

MỤC LỤC

- File Hierarchy & Loading
- Golden Rules
- Cấu trúc nên dùng
- Nên viết gì
- Anti-patterns
- Falsifiable Rules
- Progressive Disclosure
- Enforcement Pairing
- CLAUDE.md vs Hooks vs Skills vs Memory
- Starter Template
- Troubleshooting & Commands

1 File Hierarchy & Loading Order

Claude Code load **CLAUDE.md** theo thứ tự ưu tiên từ trên xuống. File ở scope cao hơn apply cho mọi project, file ở scope thấp hơn override hoặc bổ sung context cho directory cụ thể.

Managed Policy /etc/claude/CLAUDE.md · %PROGRAMDATA%\cLaude\CLAUDE.md

Org admins deploy. Không thể override bởi user hoặc project.

User ~/.cLaude/CLAUDE.md

Personal preferences — style, conventions, shell aliases. Apply mọi project.

Project ./CLAUDE.md · ./cLaude/CLAUDE.md

Commit vào repo. Shared across team. Core project context.

Local ./CLAUDE.local.md

Gitignored. Cá nhân developer — WIP notes, local paths, overrides.

LOADING BEHAVIOR

Root **CLAUDE.md** load tại launch. Subdirectory **CLAUDE.md** load **on-demand** khi Claude đọc/sửa file trong directory đó. Parent directory files cũng được tự động load khi cần.

2 Golden Rules

Consensus từ 15+ sources, bao gồm evidence-based study của Alex Dunlop trên 1,650 sessions, official Anthropic docs, và cộng đồng:

- Under 200 lines per file.** Đây là con số được nhắc đến nhiều nhất. File dài hơn → adherence giảm rõ rệt. Hard limit là 4 MiB nhưng thực tế nên giữ ngắn hơn nhiều. Context budget của CLAUDE.md chiếm ~5% total context — mỗi dòng thừa là token mất.
- CLAUDE.md là advisory, không phải enforced.** Claude đọc và cố gắng tuân theo, nhưng không có guarantee. Muốn deterministic enforcement → dùng **PreToolUse** / **PostToolUse** hooks.

- **Viết cho AI, không phải cho người.** Tránh prose dài dòng. Dùng imperative mood, bullet points, structured sections. Claude parse structured text tốt hơn paragraphs.
- **Falsifiable over vague.** "Write good code" là vô nghĩa. "All async functions must have a timeout parameter" là testable. Nếu không thể viết test cho rule → rule đó quá mơ hồ.
- **Keep it evergreen.** Không đặt task-specific instructions vào CLAUDE.md. Nó là project constitution, không phải todo list.

3 Cấu trúc nên dùng

Một `CLAUDE.md` tốt thường có các section sau, theo thứ tự ưu tiên giảm dần (important nhất ở đầu file):

```
# Project Overview
# - 2-3 câu. Đủ để Claude hiểu "đây là project gì".

# Tech Stack
# - Explicit versions. Không để Claude đoán.

# Code Conventions
# - Naming, patterns, file structure rules.

# Commands
# - Build, test, lint, deploy commands.
# - QUAN TRỌNG: bao gồm single-test command.

# Architecture Decisions
# - Những lựa chọn quan trọng và LÝ DO tại sao.

# Workflow & Process
# - Git conventions, PR process, CI.

# @imports
# - Deeper docs loaded on-demand.
```

PRIORITY MATTERS

Claude chú ý nhiều hơn đến nội dung ở đầu file. Đặt critical rules (test commands, forbidden patterns) **trước** nice-to-have conventions.

Section chi tiết: Commands

Đây là section có ROI cao nhất theo nhiều nguồn. Claude thường xuyên cần chạy build/test/lint nhưng không biết exact command nếu không được nói:

```
## Commands
- Build: `npm run build`
- Test all: `npm test`
- Test single: `npm test -- --grep "test name"` ← critical
- Lint: `npm run lint`
- Lint fix: `npm run lint:fix`
- Type check: `npm run typecheck`
- Dev server: `npm run dev` (port 3000)
```

EVIDENCE-BASED

Study trên 1,650 sessions cho thấy: việc include **single test command** giảm đáng kể số lần Claude chạy full test suite không cần thiết, tiết kiệm thời gian và tokens.

4 Nên viết gì — và không nên

✓ Nên

Project context mà AI không thể infer từ code: tại sao chọn tech X thay vì Y, business constraints, team conventions, deployment quirks, environment setup, exact commands, error patterns thường gặp.

✗ Không nên

Những thứ dễ đoán từ code: language syntax, framework basics, common patterns. Cũng không nên đặt task-specific instructions, TODO lists, hoặc temporary context.

Checklist nội dung

Category	Include	Example
Stack versions	Exact versions, runtime	Node 20 LTS, React 19, PostgreSQL 16
Naming	Conventions per layer	Components: PascalCase, hooks: use* prefix
Forbidden patterns	Things Claude must NOT do	Never use any in TypeScript. Never mock in integration tests.
Architecture	Key decisions + rationale	Monorepo with Turborepo – shared types in packages/shared
Error handling	Project-specific patterns	ALL API errors use AppError class from lib/errors
Test patterns	Framework + conventions	Vitest. Co-locate tests as *.test.ts. No snapshots.
Git workflow	Branch naming, commit style	Conventional commits. feat/fix/chore prefixes.

5 Anti-patterns

Những lỗi phổ biến nhất từ community reports:

AVOID Bloated files

Copy-paste toàn bộ coding standards, style guides, hoặc API docs vào CLAUDE.md. Điều này vượt quá context budget, Claude bắt đầu "forget" rules ở cuối file. Thay vào đó, dùng `@import` hoặc `.claude/rules/`.

AVOID Contradictory rules

CLAUDE.md nói "use Prettier" nhưng linter config dùng ESLint formatting. Hoặc root file nói "use npm" nhưng subdirectory file nói "use yarn". Claude sẽ inconsistently follow một trong hai. Dùng `/doctor prompt-audit` để detect conflicts.

AVOID Vague instructions

"Write clean code", "Follow best practices", "Be careful with security" — không actionable. Claude không biết standard nào là "best" cho project cụ thể.

AVOID Duplicating what code already says

Nếu `tsconfig.json` đã set `"strict": true`, không cần nhắc lại "Use strict TypeScript" trong CLAUDE.md. Claude đọc config files. Chỉ thêm context mà config không thể express.

AVOID Using CLAUDE.md as a todo list

Task-specific instructions ("Fix the login bug in auth.ts") không thuộc về đây. CLAUDE.md là constitution, không phải backlog. Dùng prompt trực tiếp hoặc `CLAUDE.local.md` cho WIP context.

6 Falsifiable Rules

Concept quan trọng nhất từ community consensus: mỗi rule trong CLAUDE.md phải **falsifiable** — có thể kiểm chứng bằng automated check hoặc code review.

✗ Vague (unfalsifiable)

"Write performant code"
"Handle errors properly"
"Use meaningful variable names"
"Follow SOLID principles"

✓ Specific (falsifiable)

"All DB queries must use parameterized statements"
"Every async function must have error boundary or try-catch"
"Variable names ≥ 3 chars, no abbreviations except: id, db, ctx"
"No class has more than 200 lines"

LITMUS TEST

Nếu không thể viết một grep/lint rule hoặc test case để kiểm tra rule → rule đó quá mơ hồ cho CLAUDE.md. Viết lại hoặc bỏ.

7 Progressive Disclosure

Giữ root **CLAUDE.md** ngắn gọn, delegate chi tiết vào các file chuyên biệt. Hai cơ chế chính:

@import syntax

```
# In root CLAUDE.md
@docs/architecture.md
@docs/api-conventions.md
@.claude/rules/testing.md
```

Claude load file được import khi cần context bổ sung. File import cũng tính vào context budget nhưng chỉ load on-demand thay vì always-on.

.claude/rules/ directory

Modular per-topic rules. Hỗ trợ YAML frontmatter **paths:** để scope rule cho specific files:

```
# .claude/rules/react-components.md
---
paths:
  - "src/components/**"
  - "src/pages/**"
---

- All components must be functional (no class components)
- Props interface named {ComponentName}Props
- Co-locate styles as {ComponentName}.module.css
- Export components as named exports, not default
```

```
# .claude/rules/database.md
---
paths:
  - "src/db/**"
  - "prisma/**"
---

- All migrations must be reversible
- Use transactions for multi-table operations
- Index every foreign key column
```

MONOREPO TIP

Dùng `claudeMdExcludes` trong settings để skip irrelevant ancestor CLAUDE.md files. Mỗi package trong monorepo nên có CLAUDE.md riêng với context của package đó.

8 Enforcement Pairing

Mỗi rule nên đi kèm mechanism enforcement. Không phải rule nào cũng cần hook — nhưng phải explicit về enforcement level:

Rule Type	Enforcement	Mechanism
"Never commit .env files"	HOOK	<code>PreToolUse</code> hook chặn <code>Bash(git add .env*)</code>
"Run tests before commit"	HOOK	<code>PreToolUse</code> hook on git commit → run test first
"Use camelCase for variables"	LINTER	ESLint <code>camelcase</code> rule. Claude sees lint errors.
"Prefer composition over inheritance"	ADVISORY	CLAUDE.md only. Không thể automate 100%.
"Add JSDoc to public functions"	CI	CI check with <code>eslint-plugin-jsdoc</code> .

```
# Ví dụ hook — .claude/hooks/prevent-env-commit.sh
#!/bin/bash
# PreToolUse hook: block git add of sensitive files
if echo "$TOOL_INPUT" | grep -qE '\.env|credentials|secrets'; then
  echo "BLOCKED: Cannot stage sensitive files"
  exit 2 # exit 2 = block the tool call
fi
```

KEY INSIGHT

CLAUDE.md rules mà không có enforcement nào = suggestions. Claude sẽ cố gắng tuân theo nhưng không guarantee, đặc biệt trong long sessions khi context bị compress. Tag mỗi rule với enforcement level để team biết rule nào là hard boundary.

9 CLAUDE.md vs Hooks vs Skills vs Memory

Mechanism	Khi nào dùng	Loaded	Enforcement
CLAUDE.md	Always-on project context, conventions, architecture	At launch (root) / on-demand (subdirs)	Advisory (soft)

Mechanism	Khi nào dùng	Loaded	Enforcement
Hooks	Hard boundaries — block dangerous operations	On tool call match	Deterministic (hard)
Skills	Reusable workflows — deploy, review, format	On-demand via <code>/skill</code>	Advisory (packaged)
Auto Memory	Learned patterns — Claude ghi nhớ corrections	First 200 lines at startup	Advisory (weakest)
<code>.claude/rules/</code>	File-scoped rules — load only cho matching paths	On-demand by path match	Advisory (scoped)

AGENTS.md compatibility

Claude Code đọc `AGENTS.md` khi không có `CLAUDE.md`. Nếu project dùng cả hai tools (ví dụ: OpenHands + Claude Code), có thể symlink hoặc `@import` lẫn nhau. Nội dung format tương thích — cả hai đều là Markdown với instructions.

Auto Memory system

Tách biệt với CLAUDE.md. Claude tự động ghi `MEMORY.md` và topic files vào `~/.claude/projects/<hash>/memory/`. First 200 lines / 25KB load at startup. Memory là nơi Claude học từ corrections — nếu sửa Claude nhiều lần về cùng một pattern, nó sẽ tự nhớ. Không cần (và không nên) duplicate memory content vào CLAUDE.md.

10

Starter Template

Template dưới đây synthesize best practices từ tất cả nguồn. Customize theo project, giữ tổng cộng under 200 lines:

```
# Project: [Name]

## Overview
[2-3 sentences. What this project does and who it's for.]

## Tech Stack
- Runtime: Node 20 LTS
- Framework: Next.js 15 (App Router)
- Language: TypeScript 5.6 (strict mode)
- Database: PostgreSQL 16 + Prisma 6
- Testing: Vitest + Playwright
- Package manager: pnpm 9

## Commands
- Install: `pnpm install`
- Dev: `pnpm dev` (port 3000)
- Build: `pnpm build`
- Test all: `pnpm test`
- Test single: `pnpm test -- -t "test name"`
- Lint: `pnpm lint`
- Type check: `pnpm typecheck`
- DB migrate: `pnpm prisma migrate dev`

## Code Conventions
- All code, comments, and variable names in English
- Components: PascalCase, functional only, named exports
- Hooks: `use` prefix, one hook per file
- Types: no `any`, no `as` casts, no `@ts-ignore`
- Imports: absolute paths via `@/` alias
- Error handling: all async code uses try-catch or .catch()
- No console.log in production code (use logger from `@/lib/logger`)

## File Structure
```

```

    ...
src/
  app/          # Next.js App Router pages
  components/   # Shared UI components
  lib/          # Business logic, utilities
  db/           # Prisma schema, migrations, queries
  types/        # Shared TypeScript types
  ...

## Architecture Decisions
- Server Components by default. Client Components only for interactivity.
- All data fetching in Server Components or Server Actions.
- State management: URL params + React Context. No Redux/Zustand.
- Reason: minimize client bundle, leverage streaming.

## Testing
- Co-locate test files: `Component.test.tsx` next to `Component.tsx`
- No snapshot tests
- Integration tests for API routes in `tests/integration/`
- E2E tests for critical paths in `tests/e2e/`

## Git Workflow
- Branch naming: `feat/`, `fix/`, `chore/` prefix
- Conventional Commits: `feat:`, `fix:`, `chore:`, `docs:`
- Always create new branch for changes
- Run `pnpm test && pnpm typecheck` before committing

## Deeper docs
@docs/api-patterns.md
@docs/deployment.md
@.claude/rules/
```

/INIT COMMAND

Claude Code có built-in `/init` command tự động generate starter CLAUDE.md từ project analysis. Chạy trong project root, nó sẽ scan codebase và tạo initial draft. Dùng template trên để refine output của `/init`.

11 Troubleshooting & Commands

Command	Mục đích
<code>/init</code>	Auto-generate CLAUDE.md từ project analysis
<code>/doctor prompt-audit</code>	Detect conflicts, outdated content, contradictions across instruction files
<code>/memory</code>	Xem và quản lý auto memory (MEMORY.md + topic files)
<code>/context</code>	Xem context hiện tại — bao gồm loaded CLAUDE.md files
<code>/cost</code>	Xem token usage — kiểm tra xem CLAUDE.md có chiếm quá nhiều context không

Common issues

- **Claude ignores rules ở cuối file:** File quá dài. Rút gọn, dùng `@import` hoặc `.claude/rules/`.
- **Inconsistent behavior across sessions:** Check for conflicting rules giữa các scope. Chạy `/doctor prompt-audit`.
- **Claude không biết build command:** Commands section thiếu hoặc ở quá cuối file. Move lên top.
- **Subdirectory rules không load:** Claude chỉ load subdirectory CLAUDE.md khi đọc/sửa file trong đó. Nếu cần always-on, đặt trong root hoặc dùng `@import`.
- **Monorepo: ancestor CLAUDE.md gây nhiễu:** Dùng `claudeMdExcludes` setting để exclude irrelevant parent files.

Sources

OFFICIAL Anthropic — Memory & CLAUDE.md	RESEARCH Alex Dunlop — Evidence-based CLAUDE.md (1,650 sessions)
COMMUNITY DEV Community — Multiple CLAUDE.md guides	GUIDE Techsy.io — CLAUDE.md Mastery
GUIDE BuildCamp — Claude Code Guide	GUIDE SmartScope — CLAUDE.md Best Practices
COMMUNITY AgentsRoom — Ultimate Guide	TOOL AgentLint — CLAUDE.md Linter
GUIDE Shipyard — Claude Code Tips	COMMUNITY GitHub Gist — CLAUDE.md examples
GUIDE Amit Ray — CLAUDE.md Deep Dive	COMMUNITY Skool — AI Coding Community
GUIDE UX Planet — Claude Code Workflow	GUIDE InventiveHQ — Claude Code Setup
BLOG Nikiforov — CLAUDE.md Patterns	